# Phase 02 — DNS and Domains (Complete Notes)

> *How `example.com` becomes an IP address.*

---

## 1. What is DNS and why does it exist?

Computers find each other by IP ( `142.250.190.78` ), but humans remember names ( `google.com` ). **DNS (Domain Name System)** is the global system that translates names → IPs.

**Analogy:** the phonebook of the Internet. You know the name "Dr. Rahman"; the phonebook gives you the number to dial.

Why not just use IPs?

- Humans can't memorize numbers at scale.
- IPs change (new server, new cloud provider) — the name stays stable. DNS is a layer of **indirection**, and indirection is how the Internet stays flexible. When you migrate your API from a VPS to AWS, you change one DNS record; users notice nothing.

## 2. Anatomy of a domain

```
https://api.shop.example.com.
        └┘ └─┘ └─────┘ └┘ └── (hidden) root "."
         |   |     |      └── TLD: com
         |   |     └── domain you registered: example
         |   └── subdomain: shop
         └── subdomain of subdomain: api
```

- **TLD** (top-level domain): `.com` , `.org` , `.bd` , `.dev` ...
- **Registered domain**: `example.com` — the part you buy.
- **Subdomains**: free, unlimited, you create them yourself in your DNS panel: `api.example.com` , `admin.example.com` , `staging.example.com` . This is how one domain runs many services.

### Domain registration

You *rent* a domain (yearly) from a **registrar** (Namecheap, Cloudflare, GoDaddy...). The registrar records, in the TLD's registry, **which nameservers answer for your domain**. That's the crucial link: registrar → "for example.com, ask these nameservers."

## 3. DNS architecture — the hierarchy

DNS is a distributed, hierarchical database. Nobody stores the whole phonebook; each level knows who to ask next.

```
                  ROOT SERVERS  (".", 13 logical clusters worldwide)
                  "for .com, ask the com TLD servers"
                        |
                  TLD SERVERS  (.com registry)
                  "for example.com, ask ns1.cloudflare.com"
                        |
              AUTHORITATIVE SERVERS  (YOUR nameservers)
              "example.com is 93.184.216.34"  ← the actual answer
```

Plus one more player on your side:

- **Recursive resolver** — the servant that walks the hierarchy *for* you. Usually your ISP's, or public ones: `8.8.8.8` (Google), `1.1.1.1` (Cloudflare).

## 4. Full DNS resolution, step by step

You type `example.com`:

```
1. Browser cache          — have I looked this up recently?      hit → done
2. OS cache               — same question                        hit → done
3. Recursive resolver (e.g. 1.1.1.1) — its cache?                hit → done
   ── cache miss? the resolver does the full walk: ──
4. Ask ROOT      → "ask the .com TLD servers, here they are"
5. Ask TLD .com  → "ask ns1.cloudflare.com (authoritative)"
6. Ask AUTHORITATIVE → "example.com = 93.184.216.34, valid for 300s"
7. Resolver caches it (for TTL seconds), returns IP to your OS → browser connects
```

Typically 10–100 ms on a cache miss; ~0 ms on a hit. **Caching at every layer** is what makes DNS fast, and **TTL** (time to live — how long an answer may be cached) is what makes changes slow to propagate.

**Analogy:** finding "Dr. Rahman, dentist, Dhanmondi": ask the national directory (root) → it points to the Dhaka directory (TLD) → which points to the Dhanmondi clinic reception (authoritative) → who gives the room number (IP). Next time you skip all that — you remember it (cache).

## 5. DNS record types (memorize the first four)

| Record | Maps | Example use |
|---|---|---|
| **A** | name → IPv4 | `api.example.com → 3.7.21.9` — points domain at your server. THE most important record. |
| **AAAA** | name → IPv6 | same but IPv6 |
| **CNAME** | name → another name | `www.example.com → example.com`, or your domain → `d1234.cloudfront.net` (CDN). Cannot exist on the root domain alongside other records. |
| **MX** | domain → mail servers | routes email; priority numbers = failover order |
| **TXT** | domain → free text | domain ownership proofs, SPF/DKIM anti-spam |
| **NS** | domain → its nameservers | the glue of the hierarchy |

Typical production setup for a Spring Boot app on a VPS:

```
A     example.com        →  203.0.113.10        TTL 300
A     www.example.com    →  203.0.113.10
A     api.example.com    →  203.0.113.10        (NGINX on the server routes by name — Phase 5)
MX    example.com        →  10 mail.protonmail.ch
TXT   example.com        →  "v=spf1 ..."
```

## 6. Commands — dig and nslookup

```
dig example.com                # full query; ANSWER SECTION has the A record + TTL
dig example.com +short         # just the IP
dig example.com AAAA +short    # IPv6
dig example.com MX             # mail servers
dig example.com NS             # authoritative nameservers
dig www.github.com CNAME +short

dig @8.8.8.8 example.com       # ask a SPECIFIC resolver (compare answers!)
dig example.com +trace         # ★ watch the real walk: root → TLD → authoritative

nslookup example.com           # older tool, same idea; available everywhere incl. Windows
```

Reading `dig` output: `example.com. 247 IN A 93.184.216.34` → name, remaining TTL seconds, class, record type, value.

## 7. Real-world production examples

- **Deploy day:** you point `api.example.com` (A record) at your VPS IP. Later you migrate to AWS: change the A record to the new IP; within TTL seconds the world follows.
- **TTL strategy before a migration:** days before, lower TTL 3600 → 60 so the switch propagates in a minute. After, raise it back (higher TTL = more caching = fewer lookups = faster + cheaper).
- **DNS as load balancing / failover:** Route 53 (Phase 8) can return different IPs per user location (latency routing) or stop returning a dead server's IP (health checks).

## 8. Common mistakes

- "I changed DNS but the old site still loads!" → old record cached until TTL expires. Not broken; wait, or test with `dig` directly at the authoritative server.
- Setting huge TTLs (86400) before a migration, then being stuck for a day.
- Pointing an A record at a server where nothing listens yet — DNS resolves fine, the connection then fails. DNS only finds the building; it doesn't guarantee anyone is home. *DNS problem vs server problem — always distinguish these.*
- Using CNAME at the root domain (breaks MX/NS — most DNS providers block it; use ALIAS/ANAME or an A record).
- Forgetting the `www.` record, so `www.example.com` dies while `example.com` works.

## 9. Best practices

- TTL 300 (5 min) for records you might change; raise for very stable ones.
- Always create both `example.com` and `www.example.com`.
- Use a serious DNS host (Cloudflare, Route 53) rather than the registrar's default panel.
- To debug, always compare: `dig @1.1.1.1 site.com` vs `dig @<authoritative-ns> site.com` — cached vs source of truth.

## 10. Interview questions

1. Walk through full DNS resolution for a cold cache, naming all four server types (resolver, root, TLD, authoritative).
2. A vs CNAME — when each, and why no CNAME at the root?
3. What is TTL? You migrated servers and some users still hit the old one — explain and prevent.
4. Where can DNS answers be cached? (browser, OS, resolver…)
5. Does DNS use TCP or UDP? *(UDP port 53 normally; TCP for large responses and zone transfers.)*
6. Your site "doesn't load." How do you tell DNS failure from server failure? *(dig works? → DNS fine, blame server: ping/curl the IP directly.)*

## 11. Lab

```
dig google.com +short                # 1. resolve
dig google.com                       #    note the TTL, run again, watch TTL count down = live
caching
dig example.com +trace               # 2. the real root→TLD→auth walk
dig github.com NS +short             # 3. who is authoritative for github?
dig www.github.com +short            # 4. spot the CNAME chain
dig @1.1.1.1 example.com +short      # 5. ask Cloudflare directly
dig gov.bd ANY                       # 6. explore a .bd domain
```

Optional but highly recommended (~$2–10/yr): buy a cheap domain ( `.xyz` , `.dev` ) on Namecheap/Cloudflare — we will use it for real in Phases 5, 7 and 8 (NGINX vhosts, SSL, Route 53).

## 12. ASSIGNMENT 02 (submit to Claude)

1. Run `dig example.com +trace` and annotate the output: mark which lines are root, TLD, and authoritative answers.
2. In your own words (no peeking), write the 7-step resolution story for `api.myshop.com` typed into a fresh browser.
3. Design the DNS records for this architecture: main site + `api.` subdomain on VPS `203.0.113.50` , `blog.` hosted on `myblog.hashnode.dev` (external service), email via Google Workspace. Write the exact record table (type, name, value, TTL).
4. Scenario: Monday you must migrate `api.example.com` (TTL currently 86400) to a new IP with under 5 minutes of user impact. Write your day-by-day plan.
5. Interview questions 1, 3, 6 — answer from memory in writing.